Hands-on Activity: Quantizing a Transformer Model — SOLUTION

SC 17, Module 1 — Apply Activity (25 min)

This is the complete, working solution. It uses `distilbert-base-uncased-finetuned-sst-2-english` as the checkpoint, since the course's own proprietary sample checkpoint was not available at the time this solution was built — swap `MODEL_CHECKPOINT_PATH` for the actual provided checkpoint once packaged; every other line of code is unaffected by that swap.

Environment Setup

```
conda create -n sc17-quant python=3.11 -y
conda activate sc17-quant
pip install -q ipykernel transformers "optimum-onnx[onnxruntime]" datasets
onnxruntime numpy
python -m ipykernel install --user --name sc17-quant --display-name "SC17
Quantization"
```

Select the **"SC17 Quantization"** kernel in Jupyter before running the cells below.

Task 1: Export and Quantize the Model (8 min)

In [ ]:
```python
import os
import platform
from optimum.onnxruntime import ORTModelForSequenceClassification, ORTQuantizer
from optimum.onnxruntime.configuration import AutoQuantizationConfig

# Detect CPU architecture so the same notebook runs on Apple Silicon or Intel/AMD
if platform.machine() in ("arm64", "aarch64"):
    onnx_arch = "arm64"
else:
    onnx_arch = "avx2"

MODEL_CHECKPOINT_PATH = "distilbert-base-uncased-finetuned-sst-2-english"
ONNX_OUTPUT_DIR = "onnx_fp32"
QUANTIZED_OUTPUT_DIR = "onnx_dynamic_int8"

# Load and export the checkpoint to ONNX
onnx_model = ORTModelForSequenceClassification.from_pretrained(
    MODEL_CHECKPOINT_PATH,
    export=True,
)
onnx_model.save_pretrained(ONNX_OUTPUT_DIR)

# Instantiate an ORTQuantizer from the exported ONNX model
quantizer = ORTQuantizer.from_pretrained(onnx_model)

# Build a dynamic quantization config (is_static=False -> no calibration data needed)
dynamic_config = getattr(AutoQuantizationConfig, onnx_arch)(is_static=False, per_channel=Fals

# Run quantization
quantizer.quantize(save_dir=QUANTIZED_OUTPUT_DIR, quantization_config=dynamic_config)

fp32_size = os.path.getsize(os.path.join(ONNX_OUTPUT_DIR, "model.onnx")) / (1024 * 1024)
int8_size = os.path.getsize(os.path.join(QUANTIZED_OUTPUT_DIR, "model_quantized.onnx")) / (10
print(f"FP32 ONNX model:  {fp32_size:.1f} MB")
print(f"INT8 ONNX model:  {int8_size:.1f} MB")
print(f"Size reduction:   {(1 - int8_size / fp32_size) * 100:.1f}%")
```

Answers to the practice questions:

1. Trying to quantize a model that was never exported to ONNX fails, because `ORTQuantizer` operates on an ONNX graph, not a raw PyTorch model — `from_pretrained()` expects the config metadata that only the ONNX export step produces.
2. Dynamic quantization computes activation scaling factors at inference time, for each input, so it never needs a representative calibration dataset the way static quantization does.
3. See the printed output above for the file-size comparison.

Task 2: Build the Benchmarking Loop (10 min)

In [ ]:
```python
import time
import numpy as np
import onnxruntime as ort
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(MODEL_CHECKPOINT_PATH)

SAMPLE_TEXTS = [
    "The product arrived on time and works exactly as described.",
    "Customer support was unhelpful and the issue was never resolved.",
    "Shipping was fast but the packaging was damaged.",
]

def benchmark_model(session, tokenized_inputs, warmup_iters=10, timed_iters=50):
    # Warm-up phase: untimed, lets the runtime reach a steady state
    for _ in range(warmup_iters):
        session.run(None, tokenized_inputs)

    # Timed phase: record each individual forward-pass latency
    latencies = []
    for _ in range(timed_iters):
        start = time.perf_counter()
        session.run(None, tokenized_inputs)
        latencies.append(time.perf_counter() - start)

    p50 = float(np.percentile(latencies, 50))
    p95 = float(np.percentile(latencies, 95))
    return p50, p95

def make_ort_inputs(session, text):
    inputs = tokenizer(text, return_tensors="np")
    valid_names = {i.name for i in session.get_inputs()}
    return {k: v for k, v in inputs.items() if k in valid_names}

fp32_session = ort.InferenceSession(os.path.join(ONNX_OUTPUT_DIR, "model.onnx"))
int8_session = ort.InferenceSession(os.path.join(QUANTIZED_OUTPUT_DIR, "model_quantized.onnx"))

sample_input_fp32 = make_ort_inputs(fp32_session, SAMPLE_TEXTS[0])
sample_input_int8 = make_ort_inputs(int8_session, SAMPLE_TEXTS[0])

fp32_p50, fp32_p95 = benchmark_model(fp32_session, sample_input_fp32)
int8_p50, int8_p95 = benchmark_model(int8_session, sample_input_int8)

print(f"FP32 p50: {fp32_p50 * 1000:.2f} ms   FP32 p95: {fp32_p95 * 1000:.2f} ms")
print(f"INT8 p50: {int8_p50 * 1000:.2f} ms   INT8 p95: {int8_p95 * 1000:.2f} ms")
print(f"p50 improvement: {(1 - int8_p50 / fp32_p50) * 100:.1f}%")
print(f"p95 improvement: {(1 - int8_p95 / fp32_p95) * 100:.1f}%")
```

Answers to the practice questions:

1. Skipping warm-up means the first timed call also pays for one-time costs — graph loading, memory allocation, JIT/kernel setup — that every later call skips. That makes the very first measurement artificially high and not representative of steady-state performance.
2. p95 specifically needs the full distribution of individual call times, not just a total — a single aggregate elapsed time can only ever give you a mean, and can't reveal that (for example) 5% of calls took far longer than the rest.
3. See the printed p50/p95 output above for both models.

Task 3: Write the Trade-off Report (7 min)

Example predicted-label comparison:

In [ ]:
```
for text in SAMPLE_TEXTS:
    fp32_inputs = make_ort_inputs(fp32_session, text)
    int8_inputs = make_ort_inputs(int8_session, text)
    fp32_logits = fp32_session.run(None, fp32_inputs)[0]
    int8_logits = int8_session.run(None, int8_inputs)[0]
    fp32_label = int(np.argmax(fp32_logits))
    int8_label = int(np.argmax(int8_logits))
    match = "match" if fp32_label == int8_label else "MISMATCH"
    print(f"[{match}] FP32={fp32_label} INT8={int8_label} | {text}")
```

Example trade-off report (~200 words):

The INT8 dynamic quantization pipeline reduced the exported ONNX model from its FP32 size down to roughly a quarter of the original file size — a size reduction in the 65–75% range, consistent with INT8's theoretical 4x compression ratio for weight storage. Latency improved substantially at both percentiles: p50 dropped by roughly 60–70%, and p95 improved by a comparable or slightly smaller margin, indicating the quantized model is not just faster on average but also more consistent under load, without a new long tail of slow outlier calls. Predicted labels matched between the FP32 and INT8 models on all sample inputs tested.

Given this combination — a large size reduction, a large latency improvement at both p50 and p95, and no observed change in predicted labels on the sample data — dynamic PTQ alone appears sufficient to meet the startup's real-time requirement without further investment. Static PTQ or QAT would only be worth pursuing next if a larger, more representative validation set later reveals accuracy degradation that this small sample didn't catch, or if the specific deployment hardware doesn't realize the expected latency gains from dynamic quantization in practice. As it stands, the recommendation is to ship the dynamic PTQ model and monitor production accuracy rather than investing further engineering time upfront.

(Replace the specific percentages above with your own notebook's actual printed numbers before submitting — these are illustrative, not fixed answers.)